

Tema: Banco de dados relacional para controle de clientes,
produtos e pedidos.

banco de dados e vendas

NOME: joice maria schoff dos santos

Salto veloso,08 de outubro de 2026

1. Introdução

A empresa apresentada no desafio possui informações de vendas espalhadas em planilhas. Essa situação dificulta consultas, aumenta a possibilidade de erros e torna a geração de relatórios mais demorada. Para solucionar o problema, foi proposta a criação de um banco de dados relacional em SQL.

O banco foi planejado para armazenar clientes, produtos, pedidos e os itens de cada pedido. Dessa forma, as informações ficam centralizadas e podem ser consultadas de maneira rápida e organizada.

2. Tipo de banco de dados

O projeto utiliza um banco de dados relacional, implementado em MySQL. Nesse modelo, os dados são organizados em tabelas que possuem relações entre si por meio de chaves primárias e estrangeiras.

O MySQL é adequado para esse projeto porque permite criar tabelas, relacionamentos, consultas, views, procedures e triggers, além de ser amplamente utilizado em aplicações comerciais.

MODELAGEM E ESTRUTURA DO BANCO

1. Tabela Clientes

Armazena os dados básicos dos clientes: identificador, nome, e-mail, telefone e cidade. A coluna `id_cliente` é a chave primária.

2. Tabela Produtos

Armazena os produtos comercializados pela empresa, incluindo nome, categoria, preço e quantidade disponível em estoque.

3. Tabela Pedidos

Registra cada compra realizada. A coluna `id_cliente` funciona como chave estrangeira e relaciona o pedido ao cliente responsável.

4. Tabela Itens_Pedido

Relaciona os pedidos aos produtos. Ela registra qual produto foi vendido, a quantidade e o preço praticado no momento da venda. Essa tabela é necessária porque um pedido pode conter vários produtos e um produto pode aparecer em vários pedidos.

5. Relacionamentos

Clientes 1:N Pedidos: um cliente pode realizar vários pedidos. Pedidos 1:N Itens_Pedido: um pedido pode possuir vários itens. Produtos 1:N Itens_Pedido: um produto pode aparecer em vários itens de pedidos.

6. Normalização

A estrutura evita repetir dados desnecessariamente. Os dados do cliente ficam somente em Clientes e os dados do produto ficam somente em Produtos. Os pedidos e seus itens são separados, reduzindo redundância e facilitando alterações futuras.

CONSULTAS E RESULTADOS

1. Clientes que realizaram mais compras

Foi criada uma consulta utilizando COUNT e GROUP BY para contar os pedidos de cada cliente e ordenar os resultados do maior para o menor número de compras.

2. Produto mais vendido no último mês

A consulta relaciona Produtos, Itens_Pedido e Pedidos. A função SUM calcula a quantidade vendida e o filtro de data considera os pedidos realizados no último mês.

3. Valor total de vendas por cliente

Foi utilizada a função SUM para somar os valores dos pedidos de cada cliente. O resultado permite identificar quanto cada cliente movimentou em compras.

4. Pedidos acima de determinado valor

A consulta utiliza WHERE para localizar pedidos cujo valor total seja superior a R\$ 1.000,00. O valor pode ser alterado conforme a necessidade da empresa.

5. Inserção, alteração e exclusão

O script contém comandos INSERT para cadastrar dados fictícios, UPDATE para alterar informações e DELETE para excluir registros. Essas operações representam as principais ações de manutenção dos dados.

Importância das consultas

As consultas permitem transformar os dados armazenados em informações úteis para a tomada de decisões, como identificar os clientes mais ativos, os produtos de maior saída e os pedidos de maior valor.

PROCEDURES, VIEWS, TRIGGERS E TABELA TEMPORÁRIA

1. Stored Procedures

Stored Procedures são rotinas armazenadas no banco de dados. Neste projeto foram criadas procedures para listar os pedidos de um cliente e calcular o total vendido para um cliente específico. Elas ajudam a reutilizar consultas e facilitam a execução de tarefas frequentes.

2. Views

Views são consultas armazenadas que podem ser utilizadas como se fossem tabelas virtuais. A view vw_resumo_pedidos apresenta o número do pedido, cliente, data e valor. A vw_vendas_produtos apresenta a quantidade vendida e o faturamento de cada produto.

3. Triggers

Triggers são ações automáticas executadas quando ocorre determinado evento no banco. Foi criada uma trigger para reduzir o estoque quando um item de pedido é inserido e outra para impedir uma venda quando a quantidade solicitada for maior que o estoque disponível.

4. Tabela temporária

A tabela temp_resumo_vendas é criada durante a sessão para reunir um resumo das compras por cliente. Tabelas temporárias são úteis para análises intermediárias e deixam de existir quando a sessão do banco é encerrada.

5. Benefícios

Esses recursos tornam o banco mais automatizado, organizado e seguro, reduzindo tarefas repetitivas e ajudando a manter a consistência dos dados.

CONCLUSÃO

1. Resultados alcançados

O banco de dados desenvolvido atende ao desafio proposto. Ele possui tabelas relacionadas, chaves primárias e estrangeiras, dados fictícios e consultas para análise das vendas. Também foram incluídos comandos de inserção, alteração e exclusão.

2. Aplicação prática

A estrutura pode ser utilizada como base para um sistema de vendas real. Uma aplicação poderia cadastrar clientes e produtos, registrar pedidos e utilizar as consultas para gerar relatórios de vendas e estoque.

3. Segurança e organização

A utilização de relacionamentos, restrições e triggers ajuda a manter os dados consistentes. A normalização também reduz a repetição de informações e facilita a manutenção do banco.

4. Conclusão final

A criação do banco demonstra como um sistema relacional pode organizar informações que antes estavam dispersas em planilhas. Com SQL, a empresa consegue armazenar, consultar e analisar seus dados de forma mais rápida e confiável.

O arquivo banco_loja_vendas.sql contém o script completo para criação do banco, tabelas, dados, consultas, views, procedures, triggers e tabela temporária.

5. Estrutura final

Banco: loja_vendas | Tabelas: clientes, produtos, pedidos e itens_pedido | Recursos: consultas, INSERT, UPDATE, DELETE, VIEW, PROCEDURE, TRIGGER e tabela temporária.

ESPAÇO PARA IMAGENS / CÓDIGOS

Imagem 1 – Estrutura das tabelas

01 – CRIAÇÃO DAS TABELAS

```
CREATE DATABASE loja_vendas;
USE loja_vendas;

CREATE TABLE clientes (
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    telefone VARCHAR(20),
    cidade VARCHAR(60)
);

CREATE TABLE produtos (
    id_produto INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    categoria VARCHAR(60),
    preco DECIMAL(10,2) NOT NULL,
    estoque INT NOT NULL
);

CREATE TABLE pedidos (
    id_pedido INT PRIMARY KEY AUTO_INCREMENT,
    id_cliente INT NOT NULL,
    data_pedido DATE NOT NULL,
    valor_total DECIMAL(10,2),
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);

CREATE TABLE itens_pedido (
    id_item INT PRIMARY KEY AUTO_INCREMENT,
    id_pedido INT NOT NULL,
    id_produto INT NOT NULL,
    quantidade INT NOT NULL,
    preco_unitario DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),
    FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)
);
```

Imagem 2 – Dados das tabelas

02 – INSERÇÃO DE DADOS

```
INSERT INTO clientes
(nome, email, telefone, cidade)
VALUES
('João Silva', 'joao@email.com', '499999990001', 'Salto Veloso'),
('Maria Souza', 'maria@email.com', '499999990002', 'Videira'),
('Ana Oliveira', 'ana@email.com', '499999990003', 'Joaçaba');
```

```
INSERT INTO produtos
(nome, categoria, preco, estoque)
VALUES
('Notebook', 'Informática', 3500.00, 10),
('Mouse', 'Informática', 80.00, 50),
('Teclado', 'Informática', 150.00, 30);
```

```
INSERT INTO pedidos
(id_cliente, data_pedido, valor_total)
VALUES
(1, '2026-10-01', 3580.00),
(2, '2026-10-02', 230.00),
(3, '2026-10-03', 3500.00);
```

```
INSERT INTO itens_pedido
(id_pedido, id_produto, quantidade, preco_unitario)
VALUES
(1, 1, 1, 3500.00),
(1, 2, 1, 80.00),
(2, 2, 1, 80.00),
(2, 3, 1, 150.00),
(3, 1, 1, 3500.00);
```

Imagem 3 – Consultas SQL

03 – CONSULTAS SQL

```
-- Clientes que realizaram mais compras
SELECT c.nome, COUNT(p.id_pedido) AS total_compras
FROM clientes c
JOIN pedidos p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome
ORDER BY total_compras DESC;

-- Produto mais vendido no último mês
SELECT pr.nome, SUM(ip.quantidade) AS quantidade_vendida
FROM produtos pr
JOIN itens_pedido ip ON ip.id_produto = pr.id_produto
JOIN pedidos p ON p.id_pedido = ip.id_pedido
WHERE p.data_pedido >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
GROUP BY pr.id_produto, pr.nome
ORDER BY quantidade_vendida DESC;

-- Valor total de vendas por cliente
SELECT c.nome, SUM(p.valor_total) AS total_gasto
FROM clientes c
JOIN pedidos p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome
ORDER BY total_gasto DESC;

-- Pedidos acima de R$ 1.000
SELECT *
FROM pedidos
WHERE valor_total > 1000;

-- Alteração e exclusão
UPDATE clientes
SET cidade = 'Chapecó'
WHERE id_cliente = 1;

DELETE FROM clientes
WHERE id_cliente = 3;
```

Imagem 4 – Views, Procedures e Triggers

04 – VIEWS, PROCEDURES, TRIGGERS E TABELA TEMPORÁRIA

```
-- VIEW: resumo dos pedidos
CREATE VIEW vw_resumo_pedidos AS
SELECT p.id_pedido, c.nome AS cliente,
       p.data_pedido, p.valor_total
FROM pedidos p
JOIN clientes c ON c.id_cliente = p.id_cliente;

-- VIEW: vendas por produto
CREATE VIEW vw_vendas_produtos AS
SELECT pr.nome AS produto,
       SUM(ip.quantidade) AS quantidade_vendida,
       SUM(ip.quantidade * ip.preco_unitario) AS faturamento
FROM produtos pr
JOIN itens_pedido ip ON ip.id_produto = pr.id_produto
GROUP BY pr.id_produto, pr.nome;

-- PROCEDURE: pedidos de um cliente
DELIMITER //
CREATE PROCEDURE sp_pedidos_cliente(IN cliente_id INT)
BEGIN
    SELECT * FROM pedidos
    WHERE id_cliente = cliente_id;
END//
DELIMITER ;

-- PROCEDURE: total vendido para um cliente
DELIMITER //
CREATE PROCEDURE sp_total_cliente(IN cliente_id INT)
BEGIN
    SELECT SUM(valor_total) AS total_vendido
    FROM pedidos
    WHERE id_cliente = cliente_id;
END//
DELIMITER ;

-- TRIGGER: reduzir estoque
DELIMITER //
CREATE TRIGGER trg_reduzir_estoque
AFTER INSERT ON itens_pedido
FOR EACH ROW
BEGIN
    UPDATE produtos
    SET estoque = estoque - NEW.quantidade
    WHERE id_produto = NEW.id_produto;
END//
DELIMITER ;

-- TRIGGER: impedir venda maior que o estoque
DELIMITER //
CREATE TRIGGER trg_verificar_estoque
BEFORE INSERT ON itens_pedido
FOR EACH ROW
BEGIN
    IF NEW.quantidade > (
        SELECT estoque FROM produtos
        WHERE id_produto = NEW.id_produto
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Estoque insuficiente';
    END IF;
END//
DELIMITER ;

-- TABELA TEMPORÁRIA
CREATE TEMPORARY TABLE temp_resumo_vendas AS
SELECT c.nome AS cliente,
       SUM(p.valor_total) AS total_compras
FROM clientes c
JOIN pedidos p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome;
```

